

Министерство образования Республики Беларусь
Учреждение образования «Белорусский государственный университет
информатики и радиоэлектроники»

Факультет компьютерных систем и сетей
Кафедра информатики
Дисциплина «Методы защиты информации»

ОТЧЕТ
к лабораторной работе № 3
на тему «Асимметричная криптография. Криптосистема Рабина»

Выполнил

Е. С. Кахновский

Проверил

А. В. Герчик

Минск 2024

СОДЕРЖАНИЕ

1 Цель лабораторной работы	3
2 Теоретические сведения	4
3 Результат выполнения	4
Вывод.....	6
Список использованных источников	7
Приложение А (обязательное) Листинг исходного кода	8

1 ЦЕЛЬ ЛАБОРАТОРНОЙ РАБОТЫ

Целью выполнения данной лабораторной работы является реализация программного средства шифрования и дешифрования текстовых файлов при помощи криптосистемы Рабина. Также необходимо добавить интерфейс командной строки, который позволит выбирать файл для шифрования/дешифрования, задавать ключи и режимы операций и предусмотреть обработку возможных ошибок.

2 ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ

Криптосистема Рабина – криптографическая система с открытым ключом, безопасность которой обеспечивается сложностью поиска квадратных корней в кольце остатков по модулю составного числа. Безопасность системы, как и безопасность метода RSA, обусловлена сложностью разложения на множители больших чисел. Зашифрованное сообщение можно расшифровать 4 способами. Недостатком системы является необходимость выбора истинного сообщения из 4-х возможных.

В январе 1979 года Михаэль Озёр Рабин опубликовал описание своей системы. Было доказано, что восстановление исходного текста из зашифрованного столь же трудно, как факторизация больших чисел. Система Рабина стала первой асимметричной криптосистемой, для которой было выполнено такое доказательство. Сложность восстановления связана с трудностью извлечения квадратного корня по модулю составного числа $N = p \cdot q$. Задача факторизации и задача по извлечению квадратного корня эквивалентны, то есть: зная простые делители числа N можно извлекать квадратные корни по модулю N ; умея извлекать квадратные корни по модулю N , можно разложить N на простые множители.

Система Рабина, как и любая асимметричная криптосистема, использует открытый и закрытый ключи. Открытый ключ используется для шифрования сообщений и может быть опубликован для всеобщего обозрения. Закрытый ключ необходим для расшифровки и должен быть известен только получателям зашифрованных сообщений.

Процесс генерации ключей следующий:

- 1 Выбираются два случайных числа p и q с учётом следующих требований: числа должны быть большими, числа должны быть простыми, должно выполняться условие: $p \equiv q \equiv 3 \pmod{4}$. Выполнение этих требований сильно ускоряет процедуру извлечения корней по модулю p и q ;

- 2 Вычисляется число $n = p \cdot q$;

- 3 Число n – открытый ключ; числа p и q – закрытый.

Исходное сообщение m (текст) шифруется с помощью открытого ключа – числа n по следующей формуле: $c = m^2 \pmod{n}$. Благодаря использованию умножения по модулю скорость шифрования системы Рабина больше, чем скорость шифрования по методу RSA, даже если в последнем случае выбрать небольшое значение экспоненты.

3 РЕЗУЛЬТАТ ВЫПОЛНЕНИЯ

В ходе выполнения лабораторной было реализовано программное средство шифрования и дешифрования текстовых файлов при помощи криптосистемы Рабина.

После запуска программа предлагает выбрать доступные операции: генерация ключей, шифрация / дешифрация файла и выход. При выборе операций с файлами предложен ввод путей к файлам, если вывод пустой, то программа использует пути по умолчанию. Работа программы представлена на рисунке 1.

```
Lenovo@Eugene MINGW64 /c/BSUIR/7 sem/МБИ/Labs/lab3/lab3 (dev)
$ python main.py
Chose action:
1. GENERATE KEYS
2. ENCRYPT FILE
3. DECRYPT FILE
4. EXIT
ACTION NUMBER: 1
Please input size of key: 420
Keys generated. WELL DONE!
Keys saved in 'key.txt'
Chose action:
1. GENERATE KEYS
2. ENCRYPT FILE
3. DECRYPT FILE
4. EXIT
ACTION NUMBER: 2
Please, input file path for chiper: ./input.txt
Enter the path to save the encrypted file: ./e.txt
File saved ./e.txt
Chose action:
1. GENERATE KEYS
2. ENCRYPT FILE
3. DECRYPT FILE
4. EXIT
ACTION NUMBER: 3
Enter the path to the file to decrypt: ./e.txt
Enter the path to save the decrypted file: ./decrypt.txt
The file is decrypted and saved as ./decrypt.txt
Chose action:
1. GENERATE KEYS
2. ENCRYPT FILE
3. DECRYPT FILE
4. EXIT
ACTION NUMBER: 
```

Рисунок 1 – Работа программы

ВЫВОД

В ходе данной лабораторной работы было реализовано программное средство шифрования и дешифрования текстовых файлов при помощи криптосистемы Рабина. Программа реализована на языке программирования Python, предоставляет удобный интерфейс консоли и возможность выбора производимой операции с настройкой ключей и указанием путей к файлам, хранящим исходный, зашифрованный и расшифрованный текст.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

[1] Криптосистема Рабина [Электронный ресурс]. – Режим доступа: https://ru.wikipedia.org/wiki/Криптосистема_Рабина. – Дата доступа: 03.11.2024.

[2] Rabin cryptosystem [Электронный ресурс]. – Режим доступа: https://en.wikipedia.org/wiki/Rabin_cryptosystem. – Дата доступа: 03.11.2024.

ПРИЛОЖЕНИЕ А

(обязательное)

Листинг исходного кода

```
import random
from sympy import isprime

# Генерация ключей для системы Рабина
def generate_keys(bits=512):
    while True:
        p = random.getrandbits(bits)
        if isprime(p) and p % 4 == 3:
            break
    while True:
        q = random.getrandbits(bits)
        if isprime(q) and q % 4 == 3:
            break
    N = p * q
    return (N, p, q)

def encrypt(message, N):
    m = int.from_bytes(message.encode('utf-8'), 'big')
    if m >= N:
        raise ValueError("The message is too large for the key size")
    c = pow(m, 2, N)
    return c

def decrypt(ciphertext, p, q):
    N = p * q
    m_p = pow(ciphertext, (p + 1) // 4, p)
    m_q = pow(ciphertext, (q + 1) // 4, q)
    _, yp, yq = extended_gcd(p, q)

    r1 = (yp * p * m_q + yq * q * m_p) % N
    r2 = N - r1
    r3 = (yp * p * m_q - yq * q * m_p) % N
    r4 = N - r3

    for r in [r1, r2, r3, r4]:
        try:
            decrypted_message = r.to_bytes((r.bit_length() + 7) // 8,
            'big').decode('utf-8')
            return decrypted_message
        except UnicodeDecodeError:
            continue
    raise ValueError("Decryption failed, none of the roots produced a valid message")

# Алгоритм Евклида для нахождения gcd и коэффициентов
def extended_gcd(a, b):
    if a == 0:
        return b, 0, 1
    gcd, x1, y1 = extended_gcd(b % a, a)
    x = y1 - (b // a) * x1
    y = x1
    return gcd, x, y
```



```

def read_file(file_path):
    with open(file_path, 'r', encoding='utf-8') as f:
        return f.read()

def write_file(file_path, data):
    with open(file_path, 'w', encoding='utf-8') as f:
        f.write(data)

def menu():
    print("Chose action:")
    print("1. GENERATE KEYS")
    print("2. ENCRYPT FILE ")
    print("3. DECRYPT FILE")
    print("4. EXIT")

    choice = input("ACTION NUMBER: ")
    return choice

def main():
    N, p, q = None, None, None

    while True:
        choice = menu()

        if choice == "1":
            bits = int(input("Please input size of key: "))
            N, p, q = generate_keys(bits)
            print(f"Keys generated. WEll DONE!")
            with open('key.txt', 'w') as f:
                f.write(f"{N}\n{p}\n{q}")
            print("Keys saved in 'key.txt'")

        elif choice == "2":
            if N is None:
                print("Please, generate some keys!")
                continue
            file_path = input("Please, input file path for chiper: ")
            try:
                message = read_file(file_path)
                ciphertext = encrypt(message, N)
                enc_file_path = input("Enter the path to save the encrypted
file: ")
                write_file(enc_file_path, str(ciphertext))
                print(f"File saved {enc_file_path}")
            except Exception as e:
                print(f"Error: {e}")

        elif choice == "3":
            if p is None or q is None:
                print("Please, generate some keys!")
                continue
            file_path = input("Enter the path to the file to decrypt: ")
            try:
                ciphertext = int(read_file(file_path))
                message = decrypt(ciphertext, p, q)
                dec_file_path = input("Enter the path to save the decrypted
file: ")
                write_file(dec_file_path, message)
                print(f"The file is decrypted and saved as {dec_file_path}")

```

```
        except Exception as e:
            print(f"Error {e}")

    elif choice == "4":
        print("Exit from program")
        break

    else:
        print("Restart")

if __name__ == "__main__":
    main()
```